

C# Entity Framework Cheatsheet

2026-04-08

Table of Contents

1. Ziel	1
2. Architektur	2
3. Entity Template	3
4. Fluent API	4
4.1. Configuration Template	4
4.2. DbContext Template	4
4.3. Migration Ablauf	5
5. Repository + UnitOfWork Muster	6
5.1. Contracts in Core	6
5.2. UnitOfWork in Persistence	6
5.3. Query-Regeln für gute Read-Methoden	6
6. ReadModels / Overview-Objekte	7
6.1. Typisches Query-Template	7
7. API Design Muster	8
7.1. DTO statt Entity direkt verwenden	8
7.2. Mapper-Funktionen (DTO <→ Entity)	8
7.3. Endpoint-Gruppe aufbauen	9
7.4. Request-Parameter in Minimal APIs	9
7.5. GET alle (GET /entities)	9
7.6. GET by id (GET /entities/{id})	9
7.7. POST create (POST /entities)	10
7.8. PUT update (PUT /entities/{id})	10
7.9. DELETE (DELETE /entities/{id})	11
8. Validierung (FluentValidation)	13
9. CSV Import	14

1. Ziel

Dieser Cheatsheet ist bewusst allgemein gehalten. Er hilft dir, typische Schul-/Ubungsaufgaben mit einer mehrschichtigen C#-Architektur schnell und sauber umzusetzen.

Er deckt ab:

- Projektstruktur (Core, Persistence, API, Import/Tools)
- EF Core mit Fluent API
- Repository + UnitOfWork
- REST Endpoints + Validierung
- Reporting/Overview Queries
- Import aus Dateien

2. Architektur

Core

- Entities, Enums, Value Objects
- Interfaces (Repositories, UnitOfWork)
- QueryResult/ReadModels

Persistence

- DbContext
- Entity Configurations (Fluent API)
- Repository Implementierungen
- UnitOfWork Implementierung

WebAPI

- DTOs
- Endpoint Mapping
- Validatoren
- Dependency Injection Registrierung

Import/Console

- Datei einlesen
- Daten transformieren
- Über Repositories speichern

3. Entity Template

```
public class EntityName : EntityObject
{
    public string RequiredText { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }

    public int ParentId { get; set; }
    public ParentEntity Parent { get; set; } = null!;
}
```

4. Fluent API

Nutze Fluent API fuer DB-Regeln.

4.1. Configuration Template

```
public class EntityNameConfiguration : IEntityTypeConfiguration<EntityName>
{
    public void Configure(EntityTypeBuilder<EntityName> builder)
    {
        builder.HasKey(x => x.Id);

        builder.Property(x => x.RequiredText)
            .IsRequired()
            .HasMaxLength(100);

        builder.HasOne(x => x.Parent)
            .WithMany(p => p.Children)
            .HasForeignKey(x => x.ParentId)
            .OnDelete(DeleteBehavior.Restrict);

        builder.HasIndex(x => x.RequiredText);
    }
}
```

4.2. DbContext Template

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) :
base(options) { }

    public DbSet<EntityName> EntityNames { get; set; } = null!;
    public DbSet<ParentEntity> ParentEntities { get; set; } = null!;

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.ApplyConfigurationsFromAssembly(typeof(ApplicationDbContext)
.Assembly);
    }
}
```

4.3. Migration Ablauf

Entweder über die Konsole oder über: Tools > Entity Framework Core > Add Migration / Update Database

```
dotnet ef migrations add InitialCreate --project .\Persistence\Persistence.csproj  
--startup-project .\WebAPI\WebAPI.csproj  
  
dotnet ef database update --project .\Persistence\Persistence.csproj --startup-project  
.\WebAPI\WebAPI.csproj
```

5. Repository + UnitOfWork Muster

5.1. Contracts in Core

```
public interface IEntityRepository : IGenericRepository<EntityName>
{
    Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool? onlyActive);
}

public interface IUnitOfWork : IBaseUnitOfWork
{
    IEntityRepository Entities { get; }
    IAnotherRepository Others { get; }
}
```

5.2. UnitOfWork in Persistence

```
public class UnitOfWork : BaseUnitOfWork, IUnitOfWork
{
    public UnitOfWork(ApplicationDbContext dbContext) : base(dbContext)
    {
        Entities = new EntityRepository(dbContext);
        Others = new AnotherRepository(dbContext);
    }

    public IEntityRepository Entities { get; }
    public IAnotherRepository Others { get; }
}
```

5.3. Query-Regeln für gute Read-Methoden

- Für reine Leseabfragen: `AsNoTracking()`
- Erst filtern, dann projizieren
- In Repository nur Datenlogik, keine API-Logik
- Rückgabe für Listen: `IList<T>` oder `List<T>` konsistent halten

6. ReadModels / Overview-Objekte

Nutze eigene Typen für Auswertungen statt Entities 1:1 nach außen zu geben.

```
public record EntityOverview(  
    int Id,  
    string DisplayName,  
    bool IsActive,  
    int RelatedCount  
);
```

6.1. Typisches Query-Template

```
public async Task<IList<EntityOverview>> GetOverviewAsync(string? filter, bool?  
onlyActive)  
{  
    var query = _dbContext.Entities.AsNoTracking().AsQueryable();  
  
    if (!string.IsNullOrEmpty(filter))  
        query = query.Where(x => x.Name.Contains(filter));  
  
    if (onlyActive == true)  
        query = query.Where(x => x.ActiveUntil == null || x.ActiveUntil >= DateTime  
.Today);  
  
    return await query  
        .Select(x => new EntityOverview(  
            x.Id,  
            x.Name,  
            x.ActiveUntil == null || x.ActiveUntil >= DateTime.Today,  
            x.Children.Count))  
        .OrderBy(x => x.DisplayName)  
        .ToListAsync();  
}
```

7. API Design Muster

7.1. DTO statt Entity direkt verwenden

```
public record EntityDto(int Id, string Name, string Type);
```

7.2. Mapper-Funktionen (DTO $\leftrightarrow$ Entity)

Halte Mapping zentral in der Endpoint-Klasse (oder in einer separaten Mapper-Klasse), damit Create/Update/Get konsistent bleiben.

```
private static EntityName ToEntity(EntityDto dto)
{
    return new EntityName
    {
        Id = dto.Id,
        Name = dto.Name,
        Type = dto.Type
    };
}

private static EntityDto? ToDto(EntityName? entity)
{
    if (entity is null)
        return null;

    return new EntityDto(entity.Id, entity.Name, entity.Type);
}

private static IList<EntityDto>? ToDto(IList<EntityName>? entities)
{
    if (entities is null)
        return null;

    return entities.Select(x => ToDto(x!)).ToList();
}
```

Mapper-Regeln:

- `ToDto(Entity?)` gibt `null` zurueck, wenn Entity nicht gefunden wurde
- Listen immer ueber `Select(...).ToList()` abbilden
- Im `POST` und `PUT` keine Roh-Entities aus dem Request verwenden

7.3. Endpoint-Gruppe aufbauen

```
public static void MapEntityEndpoints(this IEndpointRouteBuilder app, string
baseRoute)
{
    var route = app.MapGroup(baseRoute).WithTags("Entity");

    // route.MapGet...
    // route.MapPost...
}
```

7.4. Request-Parameter in Minimal APIs

- Path-Parameter: `"/{id:int}"` + Lambda-Argument `int id`
- Query-Parameter: `[FromQuery(Name = "filter")] string? filter`
- Body-Parameter: komplexe Typen wie `EntityDto dto`
- Services per DI: z.B. `IUnitOfWork uow`

```
route.MapGet("/overview", async (
    [FromQuery(Name = "filter")] string? filter,
    [FromQuery(Name = "onlyActive")] bool? onlyActive,
    IUnitOfWork uow) =>
{
    var data = await uow.Entities.GetOverviewAsync(filter, onlyActive);
    return Results.Ok(data);
});
```

7.5. GET alle (GET /entities)

Zweck: alle Datensätze laden (read-only).

```
route.MapGet("", async (IUnitOfWork uow) =>
{
    var dtos = ToDto(await uow.Entities.GetNoTrackingAsync());
    return Results.Ok(dtos);
})
    .WithName("GetEntities")
    .Produces<List<EntityDto>>(StatusCodes.Status200OK);
```

7.6. GET by id (GET /entities/{id})

Zweck: genau einen Datensatz per Path-Parameter laden.

```

route.MapGet("/{id:int}", async (int id, IUnitOfWork uow) =>
{
    var dto = ToDto(await uow.Entities.GetByIdAsync(id));

    if (dto is null)
    {
        return Results.Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Entity not found",
            detail: $"No entity found with ID {id}");
    }

    return Results.Ok(dto);
})
.WithName("GetEntity")
.Produces<EntityDto>(StatusCodes.Status200OK)
.ProducesProblem(StatusCodes.Status404NotFound);

```

7.7. POST create (POST /entities)

Zweck: neuen Datensatz anlegen.

```

route.MapPost("", async (EntityDto dto, IUnitOfWork uow) =>
{
    if (dto.Id != 0)
    {
        return Results.Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Invalid request",
            detail: "The ID in the request body must be 0");
    }

    var entity = ToEntity(dto);
    await uow.Entities.AddAsync(entity);
    await uow.SaveChangesAsync();

    var createdDto = ToDto(await uow.Entities.GetByIdAsync(entity.Id));
    return Results.Created($"{baseRoute}/{entity.Id}", createdDto);
})
.WithValidation<EntityDto>()
.WithName("AddEntity")
.Produces<EntityDto>(StatusCodes.Status201Created)
.ProducesProblem(StatusCodes.Status400BadRequest);

```

7.8. PUT update (PUT /entities/{id})

Zweck: bestehenden Datensatz ersetzen/aktualisieren.

```

route.MapPut("/{id:int}", async (int id, EntityDto dto, IUnitOfWork uow) =>
{
    if (id != dto.Id)
    {
        return Results.Problem(
            statusCode: StatusCodes.Status400BadRequest,
            title: "Invalid request",
            detail: "The ID in the URL does not match the ID in the request
body");
    }

    var entity = await uow.Entities.GetByIdAsync(id);

    if (entity is null)
    {
        return Results.Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Entity not found",
            detail: $"No entity found with ID {id}");
    }

    entity.Name = dto.Name;
    entity.Type = dto.Type;

    await uow.SaveChangesAsync();
    return Results.NoContent();
})
.WithValidation<EntityDto>()
.WithName("UpdateEntity")
.Produces(StatusCodes.Status204NoContent)
.ProducesProblem(StatusCodes.Status400BadRequest)
.ProducesProblem(StatusCodes.Status404NotFound);

```

7.9. DELETE (DELETE /entities/{id})

Zweck: Datensatz löschen.

```

route.MapDelete("/{id:int}", async (int id, IUnitOfWork uow) =>
{
    var entity = await uow.Entities.GetByIdAsync(id);

    if (entity is null)
    {
        return Results.Problem(
            statusCode: StatusCodes.Status404NotFound,
            title: "Entity not found",
            detail: $"No entity found with ID {id}");
    }
}

```

```
uow.Entities.Remove(entity);  
await uow.SaveChangesAsync();  
  
return Results.NoContent();  
})  
.WithName("DeleteEntity")  
.Produces(StatusCodes.Status204NoContent)  
.ProducesProblem(StatusCodes.Status404NotFound);
```

8. Validierung (FluentValidation)

Halte Regeln im API-Projekt, nicht in Endpoints verstreut.

```
public class EntityDtoValidator : AbstractValidator<EntityDto>
{
    public EntityDtoValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty()
            .MinimumLength(3)
            .MaximumLength(100);

        RuleFor(x => x.Type)
            .NotEmpty();
    }
}
```

9. CSV Import

```
using Base.Tools.CsvImport;
using Core.Contracts;
using Core.Entities;
using Import.ImportData;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Persistence;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

var builder = Host.CreateApplicationBuilder(args);

var configuration = builder.Configuration;
var connectionString = configuration.GetConnectionString("DefaultConnection")
    ?? throw new InvalidOperationException("Connection string 'DefaultConnection' not found.");

builder.Services
    .AddDbContext<ApplicationDbContext>(options =>
        options.UseSqlServer(connectionString))
    .AddScoped<IUnitOfWork, UnitOfWork>();

var host = builder.Build();

Console.WriteLine("Migrate Database");

using (var scope = host.Services.CreateScope())
{
    var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();
    await uow.DeleteDatabaseAsync();
    await uow.MigrateDatabaseAsync();
}

Console.WriteLine("Import Data");

using (var scope = host.Services.CreateScope())
{
    var uow = scope.ServiceProvider.GetRequiredService<IUnitOfWork>();

    <<<<<<< Updated upstream
    * Duplikate über eindeutige Schlüssel erkennen
    * Großen Import in mehrere SaveChangesAsync() Blöcke teilen
    * Fehler pro Zeile sammeln statt sofort abbrechen (optional)
```



```
=== Import mit Existing-Dictionary  
[source,csharp]
```

```
Console.WriteLine("Import Data");
```

```
using (var scope = host.Services.CreateScope()) { var uow =  
scope.ServiceProvider.GetRequiredService<IUnitOfWork>();
```

```
var csvData = await new CsvImport<TModel>().ReadAsync("ImportData/Source.csv");  
var cache = (await uow.Entities.GetAsync()).ToDictionary(e => e.UniqueKey, e => e);
```

```
csvData.Select(async item =>
```

```
var csvData = await new CsvImport<TModel>().ReadAsync("ImportData/Source.csv");  
var cache = (await uow.Entities.GetAsync()).ToDictionary(e => e.UniqueKey, e =>  
e);
```

```
// LINQ-Pipeline statt foreach  
var pending = csvData.Select(async item =>  
>>>>>> Stashed changes  
{  
    if (!cache.TryGetValue(item.Identifier, out var entity))  
    {  
        entity = new TEntity { PropertyA = item.ValueA, PropertyB =  
item.ValueB };  
        await uow.Entities.AddAsync(entity);  
        cache.Add(item.Identifier, entity);  
    }  
<<<<<< Updated upstream
```

```
>>>>>> Stashed changes entity.PropertyA = item.ValueA; return entity; }).ToList(); // .ToList()  
erzwingt die Iteration durch die Daten
```

```
<<<<<< Updated upstream await uow.SaveChangesAsync(); }
```

```
=====  
    await Task.WhenAll(pending);  
  
    await uow.SaveChangesAsync();  
}
```

```
Console.WriteLine("done");
```

```
>>>>>> Stashed changes
```